

BSc Thesis Project

Advancing Prescriptive Process Monitoring: A
Multi-Agent Reinforcement Learning Redesign

Jop Mörs

Department of Information and Computing Sciences
Utrecht University

First supervisor: Dr. ir. C. (Claudio) Di Ciccio (Utrecht University)

Second supervisor: Dr. ir. X. (Xixi) Lu (Utrecht University)

April 2026

Abstract

Business Process Optimization (BPO) and Prescriptive Process Monitoring (PPM) aim to make business processes more efficient by predicting potential problems and recommending actions to address them before they occur. Many existing methods use Reinforcement Learning (RL), where an algorithm learns to make better decisions through trial and error. However, these approaches typically assume a single agent with complete knowledge of the entire process, which is unrealistic in most business environments where multiple resources each see only a limited part of what is happening.

This thesis extends MASTER (Multi-Agent System for Task Execution and Resource management), a simulation environment introduced by [1] in which each resource is modelled as its own autonomous agent operating under partial observability. Building on this foundation, we redesign three key components of the original system: the reward function is replaced with an immediate skill-match signal that fires at the moment of task assignment rather than at task completion, the observation space is extended with a normalised skill-advantage signal and a queue-load fraction, and a third algorithm, Counterfactual Multi-Agent Policy Gradients (COMA), is added alongside the original MAPPO and QMIX to enable a principled comparison of centralised-critic and value-based decomposition architectures.

All three algorithms are evaluated on 2 datasets: a synthetic loan application event log containing 1,000 cases, 19 resources, and 12 activity types. And the real-world BPI12W dataset containing 8,616 cases, 52 resources and 6 activity types. The three algorithms are compared against three baselines: random assignment, always-volunteer assignment and best-median assignment. Results show that MAPPO successfully learns an effective cooperative assignment policy, closing 65 per cent of the gap between the random baseline and the best-median oracle. COMA collapses to a degenerate policy due to a volunteer’s dilemma Nash equilibrium that its decentralised Q-function cannot resolve. QMIX learns meaningful Q-values during training but fails to generalise to full-scale evaluation due to a train-test episode length mismatch that causes queue saturation among preferred agents. These findings demonstrate that centralised-critic architectures are necessary for stable convergence in cooperative volunteering coordination games, and that the immediacy of the reward signal is more consequential for learning than the choice of algorithm architecture.

Contents

1	Introduction	4
2	Background and State of the Art	7
2.1	Process Mining	7
2.2	Reinforcement Learning	8
2.3	Multi-Agent Systems and Reinforcement Learning	10
2.3.1	MAPPO	11
2.3.2	QMIX	12
2.3.3	COMA	13
2.3.4	Decentralised Partially Observable Markov Decision Processes	13
2.4	AgentSimulator	14
2.5	Reinforcement Learning for BPO and PPM	15
2.6	Multi-Agent Reinforcement Learning for BPO and PPM	15
2.7	Previous Work	16
3	Approach	19
3.1	Modelling the Environment	19
3.1.1	Environment	19
3.1.2	Agents	20
3.1.3	Action Space	20
3.1.4	Reward Function	21
3.2	Data and Preprocessing	21
3.2.1	Dataset Description	21
3.2.2	Preparing the data	22
3.3	Training and Testing	22
3.3.1	Training Procedure	22
3.3.2	Network Structure and Hyperparameters	22
3.4	Performance Analysis	23
4	Results	24
4.1	Baselines on the Loan Application Dataset	24
4.2	Baselines on the BPI12W Dataset	26
4.3	MAPPO on the Loan Application Dataset	28
4.4	MAPPO on the BPI12W Dataset	29
4.5	COMA on the Loan Application Dataset	30
4.6	COMA on the BPI12W Dataset	32
4.7	QMIX on the Loan Application Dataset	34
4.8	QMIX on the BPI12W Dataset	35
5	Discussion	37
6	Conclusion	39

1 Introduction

In many organisations, business processes are the backbone of operations: from customer service to supply chain logistics. As these processes unfold, large amounts of data are generated in the form of event logs, recording what happened, when, and by whom. Using these logs, process mining and Predictive Process Monitoring (PPM) techniques are used to make sense of past behaviour and predict future outcomes such as delays. These predictions are valuable if used correctly. But they do not tell decision-makers what to do. In complex settings, making good decisions in real time, choosing when and how to intervene, can be critical to improve performance metrics like time, cost, or success probability.

Prescriptive Process Monitoring (PPM) is a family of techniques that shifts the focus from prediction toward action. PPM recommends interventions in ongoing processes so as to optimise future outcomes according to one or more process performance indicators [2]. In a loan origination process, for instance, a PPM method might recommend a loan officer to contact a hesitating customer early, or to fast-track a credit check to avoid a downstream bottleneck. Such interventions may include reallocating resources, rerouting tasks, or sending targeted communications. The purpose is not simply to predict what might happen, but to actively change what does happen. Each intervention, however, carries its own cost. Timing matters: intervening too early may be wasteful, while intervening too late may be ineffective. Resources are finite, and the same treatment can produce very different outcomes depending on the state of the process [3], introducing a fundamental trade-off between risk and reward.

Business Process Optimisation (BPO) addresses a related but broader challenge. Modern business processes involve numerous human and automated resources working together under conditions of uncertainty and competing demands. Achieving efficient coordination in these settings is difficult, particularly when process structures are complex, resources are interdependent, and decisions must be made in real time. Traditional optimisation techniques such as linear programming or rule-based scheduling perform well in static environments but struggle to adapt when process dynamics change. Reinforcement Learning (RL) has emerged as a promising alternative for adaptive decision-making in such settings [4]. By enabling agents to learn optimal behaviour through interaction with an environment, guided by a reward signal, RL is particularly well suited to domains where explicit supervision or predefined rules are impractical.

RL has been explored for dynamic task scheduling, intelligent resource allocation, and adaptive routing in business processes [5]. However, most RL-based BPO frameworks assume a single-agent architecture in which one central entity controls all process decisions. While this simplifies the optimisation problem, it fails to capture the distributed nature of most real-world processes. Real business processes involve multiple actors, each with partial information, local objectives, and individual responsibilities. A single-agent controller cannot fully represent this structure and may become a bottleneck as system complexity grows.

Multi-Agent Reinforcement Learning (MARL) addresses this limitation by extending RL to settings where multiple agents learn together within a shared environment. Each agent observes its local state, selects actions, and receives feedback based on system outcomes. Through repeated interaction, agents develop cooperative strategies that collectively optimise overall process performance. MARL provides a natural abstraction for distributed organisational systems, where each resource, whether human or automated, can be modelled as an autonomous learning agent that coordinates with others through shared outcomes rather than centralised control.

Similarly, Multi-Agent Systems (MASs) have long been used to simulate and analyse organisational processes. MAS-based simulation allows researchers to study emergent behaviours, resource interactions, and the consequences of different decision rules under controlled conditions. However, traditional MAS frameworks rely on expert-defined heuristics or static behavioural rules. Integrating MARL into such systems replaces these hand-coded rules with learned policies, enabling agents to discover optimised assignment strategies directly from historical event log data.

Despite these developments, a research gap remains at the intersection of BPO, MAS-based simulation, and MARL. Figure 1 illustrates this gap. Multi-agent systems have been successfully used to simulate business processes, as exemplified by AgentSimulator [6]. RL has been applied to single-agent BPO and PPM tasks [5, 7]. MARL has shown promise for decentralised optimisation in manufacturing and other physical domains [8]. However, the systematic application of MARL to event log-driven business processes, where the data comes from recorded process executions rather than continuous physical sensors, remains largely unexplored. Previous work [1] takes the first step in this direction by introducing MASTER, an environment that models each process resource as a MARL agent trained on event log data. That work demonstrates that MARL can learn effective task allocation policies but leaves several important questions open, including the role of reward signal design, the suitability of different algorithm architectures for coordination games, and the behaviour of counterfactual policy gradient methods in this setting.

This thesis builds directly on MASTER to investigate how design choices in the reward function and algorithm architecture affect learning in cooperative multi-agent task assignment. Three redesigns are introduced: the reward function is replaced with an immediate skill-match signal to resolve the temporal credit assignment problem identified in [1]; the observation space is extended with signals that give each agent better information about its relative capability for the upcoming task; and a third algorithm, COMA, is added to enable a principled comparison of centralised critic and value decomposition approaches. The resulting system is evaluated on the loan application dataset used in [1] original experiments, using the same baselines and statistical evaluation protocol, to enable direct comparison.

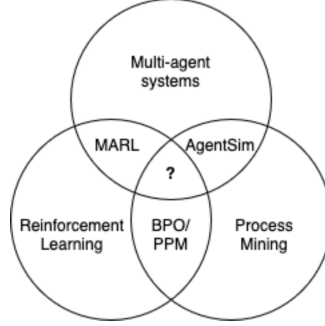


Figure 1: Venn diagram illustrating the research gap addressed by this thesis. Multi-agent systems have been used to simulate business processes (AgentSimulator), and reinforcement learning has been applied to single-agent BPO and PPM. MARL sits at the intersection of all three domains. The question mark denotes the unexplored intersection of MARL-based optimisation with event log-driven process simulation, which this thesis addresses by extending the MASTER environment with a redesigned reward function, an extended observation space, and a comparison of three MARL algorithm architectures.

The research question guiding this thesis is: *How do reward function design and algorithm architecture affect the ability of multi-agent reinforcement learning agents to learn effective task assignment policies in event log-driven business processes?*

To answer this question, the following sub-questions are addressed:

1. Does replacing a delayed completion reward with an immediate skill-match reward improve policy learning in the MASTER environment?
2. Which MARL algorithm architecture, centralised critic (MAPPO, COMA) or value decomposition (QMIX), is best suited to cooperative task assignment in business processes?
3. What structural properties of the coordination game determine whether a given algorithm converges to an effective policy or fails?

Section 2 reviews the relevant background, covering process mining, reinforcement learning, MARL algorithms, the AgentSimulator, and the prior work of [1]. Section 3 describes the method, including the environment design, datasets, training procedure, and evaluation protocol. Section 4 presents the experimental results for all three algorithms against the three baselines. Section 5 discusses the findings and their implications. Section 6 concludes and identifies directions for future research.

2 Background and State of the Art

This section presents a structured overview of the key research areas that support this thesis. This section begins with the foundations of process mining, before exploring reinforcement learning (RL) and its extension to multi-agent reinforcement learning (MARL). Then we review the main algorithms and methods, such as MAPPO, QMIX, COMA and QTRAN, that shape the technical basis of this work. Finally, the section considers the role of simulation environments, the use of RL and MARL for business process optimization (BPO), and prescriptive process monitoring (PPM), and concludes with a view on [1], which forms a direct antecedent to this research. Each subsection builds on the previous one, showing how the field has evolved from descriptive analysis of business processes toward adaptive, data-driven control through learning agents.

2.1 Process Mining

Process mining is a research field situated at the intersection of data science and business process management. Designed for industry professionals, it focuses on extracting, analyzing, and improving business processes by leveraging event logs. An event log is a record that tracks activities within a computer system, network, or application. These logs capture details like timestamps, event types, user actions, and system changes. And they are used for troubleshooting, security monitoring, and compliance. In its most basic form, process mining seeks to answer the question of how a business process actually operates, rather than how it is intended to operate. Process mining builds upon three central pillars: process discovery, conformance checking, and enhancement. Process discovery generates a process model automatically from event data. Conformance checking compares this discovered model to a reference model. Enhancement seeks to improve the model or process based on the insights gained.

The evolution of process mining from descriptive to prescriptive analytics represents a crucial turning point in business process intelligence[9, 10]. Early research focused on reconstructing what happened in a process and identifying inefficiencies. Nowadays, process mining involves using mined knowledge to guide or even automate process interventions in real time. This is where prescriptive process monitoring (PPM) comes in. A field concerned not only with predicting future events or outcomes, but also recommending actions to influence those outcomes favorably.

It is possible to integrate causal inference with reinforcement learning to determine when and how interventions should occur during process execution. [7] Effective interventions require more than correlation-based prediction because they demand causal reasoning to ensure that actions truly cause the desired changes. The approach from [7] enables prescriptive monitoring systems to balance the frequency and timing of interventions, leading to improved process efficiency and outcome quality. In this sense, the study bridges traditional process mining with intelligent, adaptive control systems that learn from feedback.

Process mining research does point out a challenge of representing processes

that combine structured and unstructured behavior. Traditional formal models, such as Petri nets or BPMN diagrams, are precise but often rigid. Informal representations, on the other hand, capture variability but sacrifice analytical rigor [11]. That is why researchers propose hybrid process models that combine both formal and informal models, allowing analysts to maintain formality where it matters while, on the other hand, accommodating flexibility. This balance between formality and adaptability mirrors the trade-offs encountered in prescriptive monitoring, where decision rules must be interpretable but also capable of evolving dynamically.

In sum, process mining lays the groundwork for understanding process behavior and identifying opportunities for intervention. Yet, as organizations increasingly seek automation and real-time decision support, the limitations of static models become visible. The next logical step is reinforcement learning, a framework that introduces adaptivity and continuous learning into process optimization.

2.2 Reinforcement Learning

Reinforcement learning (RL) is a branch of machine learning that focuses on training an agent to make decisions by interacting with an environment. The agent learns by trial and error, guided by a reward function that quantifies success in achieving specific goals. Over time, it develops a policy, a mapping from observations to actions, that maximizes cumulative reward. This method is particularly useful to dynamic systems where the consequences of actions unfold over time, making it ideal for process-oriented domains such as business operations and workflow optimization.

The relevance of RL to prescriptive process monitoring begins from its ability to handle temporary dependencies, uncertainty, and delayed feedback. Unlike predictive models, which estimate future outcomes based on current states, RL explicitly learns from the consequences of interventions. This aligns closely with the goal of prescriptive monitoring: determining not just what will happen, but what should be done to influence what happens. In the context of business processes, each activity, event, or intervention can be seen as an action that modifies the trajectory of the system.

Bozorgi et al [7] provides an early demonstration of the potential of RL for this task. By coupling reinforcement learning with causal inference, the authors ensure that the system does not just exploit correlations in data but instead learns intervention policies grounded in causality. The approach trains an agent to decide when to intervene in a running process based on contextual factors, historical behavior, and estimated causal effects. This integration of causality and learning marks a switch from static, rule-based monitoring systems.

A deeper look into reinforcement learning’s technical foundations reveals a wide variety of methods for policy learning. Among them, value-based methods like Q-learning estimate the expected return for each state-action pair, while policy-based methods such as Proximal Policy Optimization (PPO) directly optimize the policy parameters. Actor-critic methods combine these approaches,

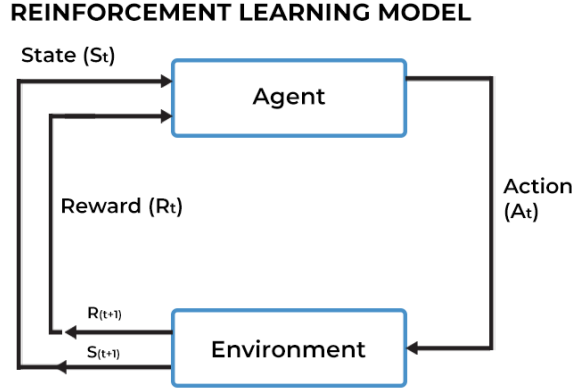


Figure 2: Visual representation of a reinforcement learning model

allowing efficient learning in high-dimensional spaces. These algorithms form the backbone of modern RL and serve as stepping stones toward more complex systems involving multiple agents.

An important development in this direction is described in [12]. Although this work primarily concerns multi-agent settings, it introduces concepts that are fundamental to understanding how RL can scale and generalize. Specifically, it proposes a method to decompose a global value function into local components associated with individual agents while preserving a monotonic relationship between them. This decomposition ensures that local improvements contribute to global performance. The link between local improvement and global performance is a property crucial for coordinated decision-making. While initially developed for multi-agent environments, these principles also apply to single-agent RL systems operating across multiple decision points or process contexts.

In practice, the usefulness of reinforcement learning lies in its adaptability. Business processes are rarely static. They evolve as organizations introduce new products, systems, and regulations. RL agents can adapt to these changes without explicit reprogramming, continually refining their policies based on observed feedback. Furthermore, reinforcement learning introduces an element of autonomy to prescriptive monitoring, allowing systems to learn effective interventions even when models of the underlying process dynamics are unavailable.

The transition from process mining to reinforcement learning represents a conceptual shift from passive analysis to active optimization. Process mining explains what happened and why, while reinforcement learning explores what can be done to improve outcomes.

A particularly important challenge in applying RL to business processes is temporal credit assignment. In long-horizon processes, the consequences of a decision, such as assigning a slow agent to a task, may not manifest until hundreds of environment steps later, when case backlogs accumulate or throughput

degrades. This temporal distance between action and outcome makes it difficult for standard policy gradient methods to attribute observed rewards to the decisions responsible for them. When the reward signal fires too far after the causal action, gradient estimates become high-variance and uninformative, and learning stalls regardless of the algorithm used. Designing reward functions that fire close in time to the decisions being optimised is therefore a critical consideration for RL in process-oriented domains.

2.3 Multi-Agent Systems and Reinforcement Learning

While reinforcement learning has proven effective for single-agent environments, many real-world systems consist of multiple decision-makers interacting within a shared context. Business processes are a prime example: they involve numerous stakeholders, departments, and automated systems, each with its own objectives and responsibilities. In such settings, the actions of one entity inevitably affect others. This complexity calls for a multi-agent reinforcement learning (MARL) approach, where multiple learning agents coordinate to achieve individual and collective goals.

In MARL, each agent learns its own policy, typically based on local observations, while a central training mechanism may coordinate learning through shared objectives or global rewards. This framework enables agents to develop cooperative or competitive strategies depending on the task structure. The paradigm reflects the decentralized nature of modern organizations. For instance, a logistics department optimizing delivery times while a sales department aims to maximize order volume. Their decisions are interdependent, and optimization at one level may create inefficiencies at another unless coordination mechanisms are in place.

A key study in this area is [12], which introduces the concept of value function decomposition under monotonicity constraints. The approach allows a joint value function, representing the overall system’s performance, to be factorized into per-agent components. Crucially, it ensures that an increase in an individual agent’s Q-value cannot decrease the joint Q-value, preserving a consistent relationship between local and global optimization. This property facilitates decentralized execution — each agent can make independent decisions using its local Q-function, while still contributing to a globally optimal outcome.

The utility of MARL in the context of business process management is twofold. First, it provides a natural abstraction for modeling distributed decision-making across organizational units. Second, it allows for adaptive coordination strategies that evolve as the process environment changes. When combined with process mining insights, which reveal dependencies and bottlenecks among process entities, MARL enables a new level of prescriptive control. Instead of a single centralized controller, multiple agents learn to cooperate in improving process performance, reflecting the distributed nature of real enterprises.

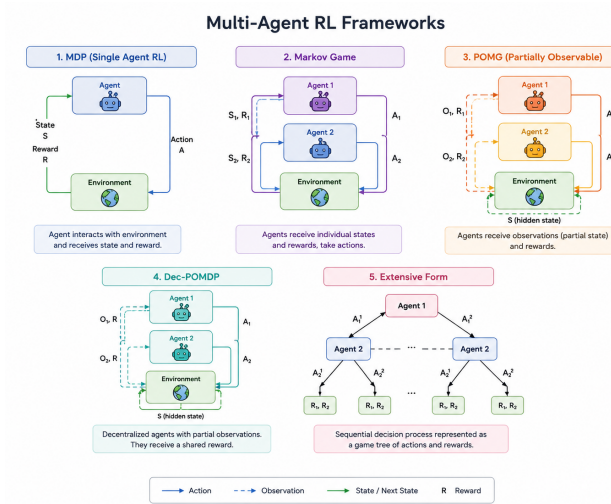


Figure 3: Visual representation of a Multi-Agent reinforcement learning model. Inspired by [13].

Multi-agent systems extend the adaptivity of reinforcement learning to collaborative and dynamic environments. The next subsections explore specific algorithms within this paradigm, notably MAPPO and QMIX, that operationalize these concepts with scalable architectures and stable training dynamics.

2.3.1 MAPPO

Multi-Agent Proximal Policy Optimization (MAPPO) is an extension of the Proximal Policy Optimization (PPO) algorithm, adapted to handle the multi-agent setting. PPO itself is a policy-gradient method that improves training stability by limiting the magnitude of policy updates. It does so through a clipped objective function that constrains how much a new policy can deviate from the old one during learning. This mechanism prevents overcorrections that could destabilize learning, a common issue in reinforcement learning when dealing with noisy rewards or sparse feedback.

MAPPO generalizes these principles to environments where multiple agents must learn simultaneously. It leverages centralized training, where a shared critic network has access to the observations and actions of all agents, while preserving decentralized execution, where each agent acts independently based on its own policy. This balance ensures efficient coordination during learning without sacrificing scalability or autonomy during deployment. Such architecture is particularly relevant for process management scenarios, where multiple subprocesses interact through shared resources or dependencies.

The usefulness of MAPPO lies in its robustness and efficiency. By maintaining a centralized critic, MAPPO can evaluate joint behaviors more ac-

curately, mitigating non-stationarity, the phenomenon where the environment changes from each agent’s perspective because other agents are also learning. At the same time, the decentralized policies ensure that agents can operate autonomously in real-world systems without relying on complete global knowledge. This dual capability makes MAPPO suitable for complex, stochastic environments where cooperation and competition coexist.

In relation to the previous subsection, MAPPO builds directly on the multi-agent paradigm by providing a practical and stable optimization method. Whereas value-decomposition methods like QMIX (discussed in the next subsection) decompose the value function to enforce coordination, MAPPO maintains coordination implicitly through shared policy gradients. Together, these two approaches, MAPPO and QMIX, represent complementary strategies for scalable multi-agent learning, both highly relevant to process optimization and adaptive decision support.

2.3.2 QMIX

QMIX, introduced in [12], is one of the most influential algorithms in the MARL literature. It is specifically designed for scenarios where agents must coordinate to maximize a shared team reward. The core idea of QMIX is to combine the individual Q-values of agents through a mixing network that enforces a monotonic relationship between the global Q-value and the local Q-values. This ensures that optimizing each agent’s local policy also contributes positively to the global objective, even though agents act independently.

QMIX achieves centralized training with decentralized execution, a principle central to many MARL algorithms. During training, a central network has access to all agents’ observations and actions, allowing it to compute the joint Q-function accurately. However, once training is complete, each agent can act independently based on its local Q-function. This design elegantly solves the coordination problem that arises in multi-agent environments, where decentralized agents must still behave coherently.

The value of QMIX lies in its capacity to scale coordination without explicit communication. For business process management, this translates into the ability to model interactions among subprocesses, departments, or actors that operate semi-independently but share organizational goals. For example, a production process might involve parallel workflows — quality assurance, logistics, and assembly, each making independent scheduling decisions that collectively impact throughput. QMIX’s value decomposition enables learning policies that optimize these distributed activities without requiring full synchronization at every decision point.

QMIX also connects naturally to MAPPO: while both aim to handle coordination in MARL, they differ in methodology. MAPPO relies on shared gradient updates and centralized critics, emphasizing training stability and continuous adaptation. QMIX, on the other hand, prioritizes structured value decomposition, ensuring interpretability and efficient execution. In combination, they provide a diverse methodological foundation for designing learning systems ca-

pable of adaptive and explainable process optimization.

2.3.3 COMA

Counterfactual Multi-Agent Policy Gradients (COMA), introduced by [14] is a policy-gradient-based MARL algorithm explicitly designed to address the multi-agent credit assignment problem. In cooperative environments with shared rewards, it is often unclear how much each individual agent contributed to the global outcome. COMA solves this problem by using counterfactual reasoning to isolate an agent’s marginal contribution to team performance.

The defining feature of COMA is its centralized critic, which estimates a joint action-value function over the actions of all agents. Instead of directly using this value to compute policy gradients, COMA constructs a baseline for each agent by marginalizing out that agent’s action while keeping the actions of all other agents fixed. The resulting advantage function measures how much better the chosen action performed compared to alternative actions the agent could have taken, given the behavior of its teammates. This mechanism enables more precise and stable policy updates by reducing variance and improving credit assignment.

Like MAPPO and QMIX, COMA follows the centralized training with decentralized execution. During training, the critic has access to global state information and joint actions, but during execution, each agent relies solely on its own policy and local observations. This design makes COMA applicable to real-world distributed systems where global information may be unavailable or impractical at runtime.

In the context of process management and organizational workflows, COMA is particularly relevant when outcomes depend on tightly coupled decisions made by multiple actors. For instance, delays in a business process may come from subtle interactions between scheduling, resource allocation, and approval decisions. COMA’s counterfactual framework allows the learning system to attribute responsibility more accurately, making it a good choice for understanding agent contributions and optimizing overall performance.

Unfortunately, COMA is computationally demanding due to its reliance on a centralized critic over joint action spaces. As the number of agents or possible actions grows, this complexity can limit scalability. Consequently, while COMA offers strong theoretical advantages for credit assignment, it is best suited to environments with moderate agent counts and well-defined interaction structures.

2.3.4 Decentralised Partially Observable Markov Decision Processes

The theoretical framework that underpins multi-agent environments with partial observability is the Decentralised Partially Observable Markov Decision Process (Dec-POMDP) [15] 3. A Dec-POMDP extends the single-agent POMDP to a multi-agent setting by distributing both the decision problem and the observational limitations across a team of agents. Formally, a Dec-POMDP is defined

by a tuple $\langle I, S, \{A_i\}, T, R, \{\Omega_i\}, O, \gamma \rangle$, where I is the set of agents, S is the global state space, A_i and Ω_i are the action and observation spaces of agent i respectively, T is the state transition function, R is the shared reward function, O is the observation function mapping global states to per-agent observations, and γ is the discount factor.

The key property of Dec-POMDPs is that no individual agent has access to the full global state $s \in S$ during execution. Each agent i receives only a local observation $o_i \in \Omega_i$, which provides a partial and potentially noisy view of the environment. Agents must therefore learn policies $\pi_i : \Omega_i \rightarrow A_i$ that are conditioned solely on local information, even though the quality of the joint outcome depends on the collective behaviour of all agents.

Dec-POMDPs are well suited to modelling resource assignment in business processes because they naturally capture the information asymmetry present in real organisations. A clerk processing a loan application knows their own workload and performance history but cannot directly observe the queue lengths or current assignments of their colleagues. The Dec-POMDP framework formalises this partial observability and motivates the Centralised Training with Decentralised Execution paradigm used by MAPPO, COMA, and QMIX: during training, a central mechanism can access the global state to improve learning, while at execution time each agent acts on its own local observation alone, reflecting how resources actually operate in practice.

2.4 AgentSimulator

To develop and evaluate reinforcement learning and MARL methods for process optimization, simulation environments play an indispensable role. An AgentSimulator provides a controlled sandbox where agents can interact with synthetic yet realistic representations of business processes. Such simulators reproduce event logs, process variations, and resource constraints, enabling safe experimentation before deployment in live systems.

Agent-based simulation allows researchers to model complex process behaviors, such as queues, delays, or stochastic outcomes, that are difficult to capture analytically. Each agent in the simulator represents an entity, such as a human worker, a software robot, or an organizational unit, interacting with the process environment and with other agents. Through repeated simulation runs, RL algorithms can explore the effects of different intervention strategies, learn from feedback, and generalize across process scenarios.

The usefulness of simulation extends beyond training; it also supports validation and benchmarking. Because reinforcement learning relies heavily on exploration, testing algorithms directly in production environments is often infeasible or risky. An AgentSimulator mitigates this by providing a reproducible environment for measuring algorithmic performance under controlled conditions. Also, simulation helps in aligning the process mining and reinforcement learning perspectives: it connects mined process models with executable environments that enable learning and evaluation.

In relation to the previous subsections, the simulator functions as a bridge

between theory and practice. Algorithms like QMIX and MAPPO provide the mathematical framework for coordination and learning, but the AgentSimulator offers the infrastructure to test these mechanisms on realistic process dynamics. Without simulation, it would be impossible to assess how learned policies behave under process variability, resource contention, or partial observability, all key features of real-world business systems.

2.5 Reinforcement Learning for BPO and PPM

The integration of reinforcement learning into business process optimization (BPO) and prescriptive process monitoring (PPM) represents a convergence of data-driven analytics and adaptive decision-making. Traditional process optimization methods rely on predefined rules, optimization heuristics, or simulation-based experimentation. In contrast, RL enables systems to learn optimal intervention strategies autonomously, guided by feedback from process performance indicators.

A notable study in this domain is [16]. The authors propose a framework that augments event logs with enriched contextual data and predictive models capable of estimating the effects of interventions. In section 3.1.1, they introduce methods for event log enrichment, predictive modeling, and naive intervention strategies. More importantly, they extend prescriptive monitoring by integrating conformal prediction, a statistical method for quantifying uncertainty in model predictions. This allows the system to estimate not just what action to take, but also how confident it should be in that recommendation.

Reinforcement learning naturally complements this framework. Whereas Shoush and Dumas focus on confidence estimation and static intervention rules, RL provides the machinery for sequential decision-making and long-term optimization. When combined, these methods enable uncertainty-aware learning systems that adapt interventions dynamically as new process events unfold. This fusion enhances both the robustness and reliability of prescriptive monitoring, addressing one of the core limitations highlighted in earlier surveys like [17].

By extending prescriptive monitoring with reinforcement learning, the process management field moves toward a paradigm of continuous optimization. Business processes become self-improving systems that learn from experience, adapting their behavior to achieve organizational goals more effectively. The next subsection extends this idea into the multi-agent setting, where multiple learning entities collaborate to optimize complex, interdependent processes.

2.6 Multi-Agent Reinforcement Learning for BPO and PPM

Multi-agent reinforcement learning (MARL) provides an even more expressive framework for business process optimization and prescriptive monitoring. In large organizations, process execution is inherently distributed: various actors, systems, and subprocesses operate concurrently, often with limited visibility of

each other’s actions. MARL captures this distributed structure by representing each process component as an autonomous learning agent.

In a MARL framework for BPO or PPM, each agent can be responsible for a specific aspect of the process, for instance, task scheduling, resource allocation, or exception handling. Agents learn to optimize their local objectives while also contributing to global process performance. Coordination among agents emerges either through shared rewards or through structured learning architectures such as QMIX or MAPPO. These systems enable process-level optimization that reflects both autonomy and cooperation, mirroring the structure of complex socio-technical organizations.

The application of MARL to business process monitoring builds directly on the insights of earlier sections. Process mining provides the data foundation; reinforcement learning provides adaptive control. And multi-agent systems introduce distributed intelligence. Together, they form a holistic approach to prescriptive analytics, one capable of capturing interdependencies, responding to feedback, and adapting to change. This integrated perspective sets the stage for recent work that applies MARL methods directly to prescriptive monitoring tasks, such as the MSc research by [1].

2.7 Previous Work

The most direct antecedent to this thesis is [1], which introduces MASTER (Multi-Agent System for Task Execution and Resource management), the first simulation environment to apply MARL to event log-driven business process optimisation. MASTER occupies the unexplored intersection of the Venn diagram shown in Figure 1: it combines agent-based simulation in the style of AgentSimulator with reinforcement learning, applying the combination specifically to BPO and PPM. This thesis extends and redesigns key components of MASTER, making [1] work both the technical foundation and the primary point of comparison for the contributions presented here.

Environment Design

MASTER models each human resource in a process as an autonomous agent operating within a Dec-POMDP. The environment is implemented as a PettingZoo parallel environment [18], in which all agents act simultaneously at each discrete time step. At each step the environment presents the upcoming task to all agents, collects their binary volunteering actions, resolves the assignment by selecting uniformly at random among capable volunteers, and advances simulated time to the next event. Task durations are sampled from statistical distributions fitted to each agent’s historical performance in the training data, rather than replaying the original timestamps. This design choice reduces the complexity of the action space and keeps the learning objective focused on assignment quality rather than process discovery.

Each agent’s local observation includes the identity of the upcoming task, the agent’s own historical mean, median, and standard deviation of processing time

for that task, the number of tasks in the agent’s queue, the remaining duration on the agent’s current assignment, and a binary flag indicating capability. Tasks that exceed a minimum length threshold of three activities are retained; shorter cases are removed to avoid policies that trivially learn to exploit degenerate traces.

Reward Function

The reward function used in MASTER [1] is a simple linear penalty on task throughput time:

$$f(d) = -d \quad (1)$$

where $d = t_{\text{end}} - t_{\text{assigned}}$ is the throughput time of the completed task in seconds. This function penalises longer task executions proportionally and is defined to be numerically stable across datasets with different duration scales. The reward fires when a task completes, not when an assignment decision is made, introducing a temporal gap between the volunteering action and the resulting reward signal.

Algorithms and Datasets

The MASTER implementation [1] provides an evaluation of two algorithms, MAPPO and QMIX, across three publicly available datasets: the semisynthetic CVS Pharmacy dataset (6 resources, 15 activities, 10,000 traces), the synthetic loan application dataset (19 resources, 12 activities, 1,000 traces), and the real-world BPI12W dataset [19] (52 resources, 6 activities, 8,616 traces). Performance is measured using throughput, waiting, and processing time distributions over 10 evaluation runs, with statistical comparisons made against three baselines: random assignment, best-median assignment, and ground-truth assignment. Non-parametric tests, including the Kruskal-Wallis H-test and pairwise Mann-Whitney U-tests with Bonferroni correction, are used throughout because the metric distributions are consistently non-normal.

Limitations and Open Questions

[1] identifies several limitations that directly motivate the redesign presented in this thesis. First, the linear throughput reward in Equation 1 fires at task completion rather than at the moment of assignment, creating a temporal credit assignment problem in episodes of 700–800 steps. The policy gradient signal is too distant from the responsible volunteering action to be attributed correctly, leading to high-variance advantages and slow convergence. Second, only two algorithms, MAPPO and QMIX, are evaluated. The counterfactual multi-agent policy gradient algorithm COMA, which is specifically designed to resolve the credit assignment problem in cooperative multi-agent settings, is absent from the comparison. Third and finally, the observation space does not include a normalised skill-advantage signal comparing each agent’s historical performance to

the global median for the upcoming activity, limiting the information available to agents for making capability-aware volunteering decisions.

This thesis addresses each of these limitations directly. The reward function is redesigned to fire immediately at assignment time, the observation space is extended with a skill-advantage signal and a queue-load fraction, COMA is added as a third algorithm to enable a principled comparison of critic architectures, and the experimental results are analysed through the lens of cooperative game theory to explain the structural causes of convergence and failure in the volunteer coordination setting.

3 Approach

This section describes the method and experimental design used to answer the research question. It is divided into the following parts:

1. Modelling the environment
2. Data and preprocessing
3. Training and testing
4. Performance analysis

The approach models each human resource in the process as an autonomous reinforcement learning agent that learns when to volunteer for upcoming tasks. By learning these volunteering decisions, the goal is to obtain an optimised assignment policy in which capable agents self-select based on their historical task-level performance, reducing overall task processing times across the process. The way MASTER-X works is visible in Figure 4.

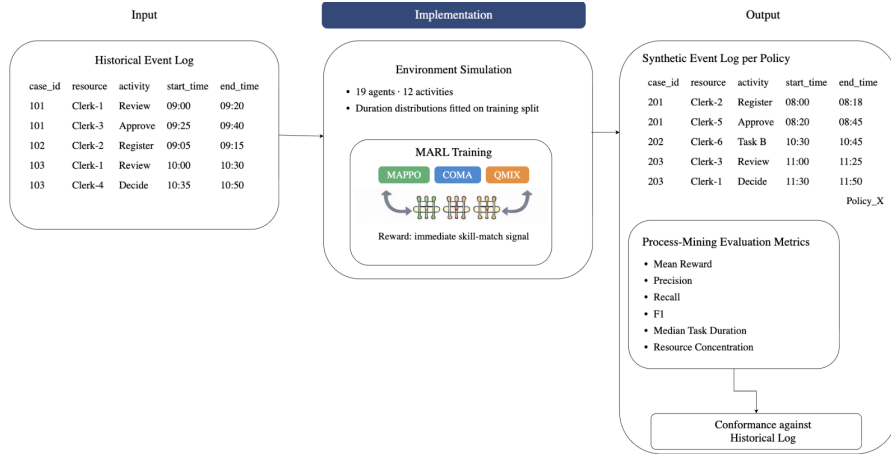


Figure 4: A schematic overview of the research approach. Event logs and process knowledge serve as input to a discrete-event simulation environment in which MARL agents are trained. The output is a set of synthetic event logs and evaluation metrics for each learned policy.

3.1 Modelling the Environment

3.1.1 Environment

The environment is structured as a Decentralised Partially Observable Markov Decision Process (Dec-POMDP). Agents make their volunteering choices autonomously with only partial information available, such as their own performance statistics for the upcoming task and the current length of their task

queue. A global state, the concatenation of all agents’ observations, is available to centralised critics during training but is not accessible to individual actors during execution, following the Centralised Training with Decentralised Execution (CTDE) paradigm.

The environment is made up of a set of cases C , where a case $c \in C$ is a sequence of tasks that, when completed, constitute a full process instance. Each task t carries three timestamps: assigned time, starting time, and completion time. Tasks and cases have one of four statuses: pending, open, in progress, and completed. At each discrete simulation step, the environment presents the upcoming task to all agents, collects their volunteering actions, resolves the assignment, and advances simulated time to the next event, either a task completion or a new case arrival.

The environment only samples completion times for each task from the assigned agent’s historical performance distribution. This reduces the complexity of the action space and enables the approach to focus on assignment quality rather than process discovery.

3.1.2 Agents

Each agent $a \in A$ represents a single human resource in the process. Agents are heterogeneous: each carries a distinct set of capabilities a_c , comprising the activities the agent has historically performed, together with a fitted duration distribution for each activity. These distributions capture the empirical variability in individual task execution times.

Each agent receives a local observation at every decision step encoding: the normalised identity of the upcoming task; the agent’s own historical mean, median, and standard deviation of processing time for that task relative to the global process scale; the remaining duration on the agent’s current assignment; a normalised representation of the tasks already queued for the agent; a binary flag indicating whether the agent is currently busy; a skill-advantage signal in $[-1, +1]$ expressing how the agent’s historical median compares to the global median for this activity; and a queue-load fraction indicating proximity to capacity. Together these signals provide sufficient context for each agent to make an informed volunteering decision without requiring knowledge of other agents’ private states.

The learned policy π_a replaces hand-coded handover behaviour used in rule-based simulation approaches. Agents work on one task at a time. If an agent volunteers for a new task while already occupied, the new task is added to the end of the agent’s task queue and processed in First-In-First-Out order.

3.1.3 Action Space

The action space is binary. At each decision step, each agent selects either action 0 (pass) or action 1 (volunteer) for the upcoming task. When multiple agents volunteer, the assigned agent is selected uniformly at random from the capable volunteers. When no agent volunteers, an agent is selected uniformly

at random from all capable agents, and a small negative penalty is applied to each capable non-volunteering agent to maintain pressure towards cooperative behaviour. This fallback ensures the process never deadlocks while the penalty discourages systematic free-riding.

3.1.4 Reward Function

The reward signal is designed to incentivise agents to volunteer selectively based on their relative skill at the upcoming task. At the moment of assignment, the selected agent receives an immediate reward:

$$R_{\text{assign}} = -\tanh\left(2 \cdot \frac{\tilde{d}_i - \tilde{d}_{\text{global}}}{\tilde{d}_{\text{global}}}\right) \quad (2)$$

where $\tilde{d}_i$ is the assigned agent’s historical median processing time for the activity and $\tilde{d}_{\text{global}}$ is the global median across all agents for that activity. This produces a reward of approximately +1 when the selected agent is substantially faster than average, 0 when they match the average, and approximately −1 when they are substantially slower. The tanh function bounds the reward in $(-1, +1)$, providing numerical stability across activities with very different duration scales.

The reward formulation draws on relative performance evaluation theory [20], reward shaping principles from reinforcement learning [21], and robust statistical estimation using the median [22]. The tanh function is used as a standard bounding mechanism.

3.2 Data and Preprocessing

3.2.1 Dataset Description

The experiments are conducted on a loan application dataset (list dataset source), a synthetic event log from a bank’s loan application process. The dataset contains 7,492 activity records across 1,000 unique cases, 19 distinct resources, and 12 activity types. An overview of the dataset statistics is shown in Table 1.

Name	Resources	Activities	Traces	events
Loan application	19	12	1,000	7,492
BPI12W	52	6	8,616	150,000

Table 1: Overview of Loan application dataset

Two datasets are used to evaluate the three algorithms. The loan application dataset [dataset’loan] provides a tractable initial benchmark with 19 resources and 12 activities. The BPI12W dataset [19], drawn from the BPI Challenge 2012, models the work-item subprocess of a Dutch financial institution’s loan origination process. It contains 8,616 traces across 52 distinct resources and 6 activity types, providing a substantially more challenging coordination problem: the larger agent population amplifies the volunteer’s dilemma, while the small

activity vocabulary means that many resources are capable of the same tasks, increasing competition at each assignment step. Both datasets are preprocessed identically: cases with fewer than three tasks are removed, and only the five required columns (case identifier, resource name, activity label, start timestamp, end timestamp) are retained.

3.2.2 Preparing the data

Cases with fewer than three tasks are removed from the dataset. This prevents agents from learning policies biased by degenerate short traces that do not represent realistic process behaviour. Only the columns relevant to the simulation are retained: case identifier, resource name, activity label, start timestamp, and end timestamp.

3.3 Training and Testing

3.3.1 Training Procedure

Three algorithms are evaluated: MAPPO, COMA, and QMIX. Each algorithm is trained from scratch with independent random seeds on the training split. To manage computational costs, training episodes use subsampled batches of 50 cases drawn randomly per episode, keeping each episode to approximately 375 environment steps. Evaluation always uses the full held-out test set without subsampling, ensuring that reported metrics reflect performance on the complete test split.

Training uses patience-based early stopping: if the 50-episode rolling average reward does not improve for 60 consecutive episodes, training terminates and the best checkpoint is restored. Models are saved whenever the rolling average achieves a new best. All algorithms run for a maximum of 300 episodes.

For BPI12W, the test split contains approximately 1,720 cases. Event logs for trained agents are generated using a fixed episode of 200 randomly sampled test cases, consistent with the training episode size. Baseline event logs are generated over the full test split to provide complete throughput statistics; direct per-task time comparisons between baselines and trained agents on BPI12W should therefore be interpreted with the episode-size caveat in mind.

3.3.2 Network Structure and Hyperparameters

Training hyperparameters are listed in Table 2. Values were selected based on preliminary experimentation and held constant across all three algorithms where applicable to support fair comparison. No formal hyperparameter search was performed, as the goal is to evaluate the feasibility of the approach rather than to maximise performance on any specific dataset.

Table 2: Hyperparameters

Parameter	MAPPO	COMA	QMIX
Learning rate (actor)	3×10^{-4}	3×10^{-4}	—
Learning rate (critic/Q)	5×10^{-4}	1×10^{-4}	5×10^{-5}
Discount factor	0.99	0.99	0.99
Actor hidden size	256	256	256
Critic/mixing hidden size	512	512	64
Hidden layers	3	3	3
PPO clip ϵ	0.2	—	—
TD- λ	0.95 (GAE)	0.8	—
Replay buffer	—	—	10,000
Batch size	128	—	128
Target network τ	—	0.005	—
Update frequency	per episode	per episode	every 20 steps
Early stopping patience	60	60	60

3.4 Performance Analysis

Performance is evaluated by comparing the task processing time distributions produced by each learned policy against three baselines. Task processing time is defined as the difference between the end time and the start time of each task ($t_{\text{end}} - t_{\text{start}}$). The mean episode reward, reported directly from the reinforcement learning training loop, serves as the primary optimisation signal. Task processing time statistics, specifically mean, median, standard deviation, minimum, and maximum, serve as the process-level performance metrics, following standard practice in business process simulation evaluation.

Three baselines are defined. The **Random** baseline assigns each task to a uniformly sampled capable agent. The **AlwaysVolunteer** baseline instructs every agent to volunteer at every step, resulting in a uniformly sampled capable volunteer being assigned. The **BestMedian** baseline always assigns the capable agent with the lowest historical median processing time for the upcoming activity, serving as an approximate oracle representing the upper bound of a statistics-driven heuristic.

All baselines are evaluated in the same simulation environment as the trained agents, using the same fitted duration distributions, to ensure a fair comparison. Resource concentration, defined as the proportion of assignments routed to the top two agents, is reported as a secondary metric to characterise the diversity of assignment policies. Oracle alignment, measured as precision, recall, and F1 against the BestMedian oracle agent, is reported to quantify how closely each policy approximates the statistically optimal assignment rule.

4 Results

This chapter reports the experimental results obtained by applying the proposed approach, MASTER-X <https://github.com/JopMors/MASTER-X/tree/main> where all results come from and are able to be replicated, to two datasets: a synthetic dataset and a real-world event log. Performance and efficacy are evaluated quantitatively in terms of process execution time and related metrics, followed by a qualitative investigation of notable cases drawn from the real-world dataset.

4.1 Baselines on the Loan Application Dataset

We first establish the performance range within which learned policies are situated. Figure 5 shows the mean episode reward for each baseline. BestMedian achieves a mean episode reward of 8.50, AlwaysVolunteer 0.06, and Random -1.88 . The large gap between BestMedian and the two uninformed baselines reflects the strong performance advantage of concentrating all assignments on a single fast resource: BestMedian routes every task to one specific agent, the statistically fastest capable agent across all activities in the training split.

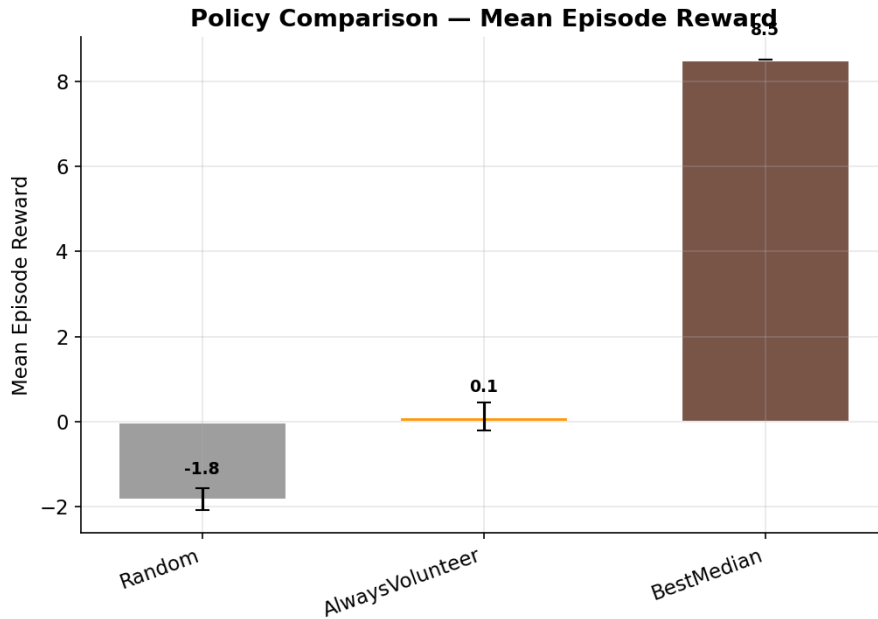


Figure 5: Mean episode reward for the three baseline policies evaluated on the 200-case test split of the loan application dataset. Error bars indicate one standard deviation across evaluation episodes.

Table 3 reports task processing time statistics for all baselines and trained agents evaluated over the full test split.

Table 3: Task processing time ($t_{\text{end}} - t_{\text{start}}$) in minutes

Agent	Mean	Median	Std Dev	Min	Max
Random	75.57	18.22	182.38	0.34	1935.63
AlwaysVolunteer	59.99	18.20	131.79	0.24	1197.39
BestMedian	43.01	15.41	68.28	0.34	254.91
MAPPO	76.71	13.43	242.56	0.10	2187.91
COMA	80.95	17.30	232.83	0.07	1819.80
QMIX	42.96	12.54	98.04	0.18	873.51

The baselines confirm that BestMedian is a strong upper bound achieved at the cost of complete load imbalance. Random and AlwaysVolunteer produce near-identical resource distributions, spreading work uniformly across all eight capable clerks, which confirms that all-volunteer behaviour is equivalent to random selection in this process.

4.2 Baselines on the BPI12W Dataset

The BPI12W baselines establish a performance range that is qualitatively similar to the loan application dataset but dramatically larger in scale. Figure 6 shows the mean episode reward for each baseline evaluated on the full test split. BestMedian achieves a mean episode reward of 230.1, while Random and AlwaysVolunteer obtain -23.3 and -24.9 respectively. The gap of approximately 253 reward units between BestMedian and the two uninformed baselines is nearly 25 times wider than on the loan application dataset, reflecting the combined effect of the larger number of cases per episode (1,720 versus 200 on loan app) and the highly concentrated skill structure of the dataset.

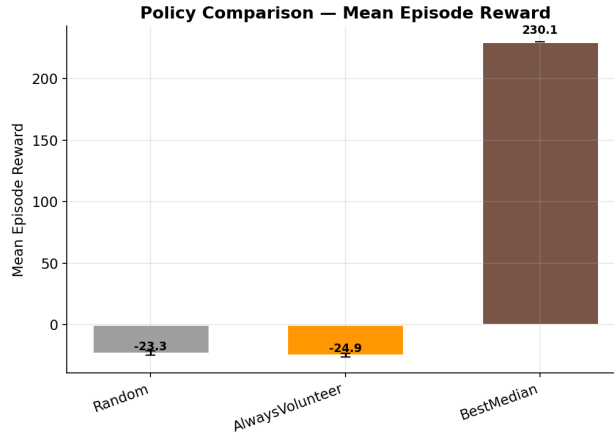


Figure 6: Mean episode reward for the three baseline policies evaluated on test split of the BPI12W dataset. Error bars indicate one standard deviation across evaluation episodes.

Table 4 reports task processing time statistics for all policies on BPI12W.

Table 4: Baseline and MARL performance on BPI12W

Agent	Cases	Events	Resources	Median	Mean	Std Dev	Min	Max	Top 2 conc.
Random	1,265	11,282	46	2.19	135.16	1,650.85	0.02	31,612.86	5.2%
AlwaysVolunteer	1,265	11,281	46	2.21	171.76	1,822.40	0.02	31,612.86	5.6%
BestMedian	1,265	11,279	6	0.35	3.29	31.02	0.02	1,991.29	68.1%
MAPPO	200	1,715	43	1.22	100.76	1,099.21	0.02	28,385.02	12.6%
COMA	200	1,747	45	2.25	146.51	1,605.02	0.02	31,612.86	6.2%
QMIX	200	1,728	27	2.49	58.63	981.81	0.02	28,385.02	14.3%

The BestMedian baseline is an exceptionally strong oracle on this dataset. It concentrates all assignments on only 6 of 52 resources (11.5%) and achieves a median task processing time of 0.35 minutes, roughly six times faster than the uninformed baselines at 2.19–2.21 minutes. This reflects the heavily skewed competency structure of BPI12W: a small group of resources is dramatically faster than the majority for every activity type, so a statistics-driven oracle can

achieve near-optimal throughput by ignoring the slow majority entirely. The mean processing time remains high for all three baselines (3.29–171.76 minutes) due to the presence of extremely long-duration outlier tasks that inflate the mean without affecting the median. AlwaysVolunteer achieves a slightly higher mean than Random (171.76 versus 135.16 minutes), suggesting that routing tasks uniformly among a larger pool of slower agents actively increases average waiting time compared to random selection.

4.3 MAPPO on the Loan Application Dataset

The MAPPO training curve, shown in Figure 7, shows rapid convergence. The rolling episode reward rises from -1.0 to a plateau of approximately 5.0 – 5.5 within the first fifteen training episodes and sustains this level with a narrow confidence band for over one hundred episodes before early stopping triggers at episode 124. This pattern indicates that MAPPO finds a stable assignment policy early and does not deteriorate over continued training.

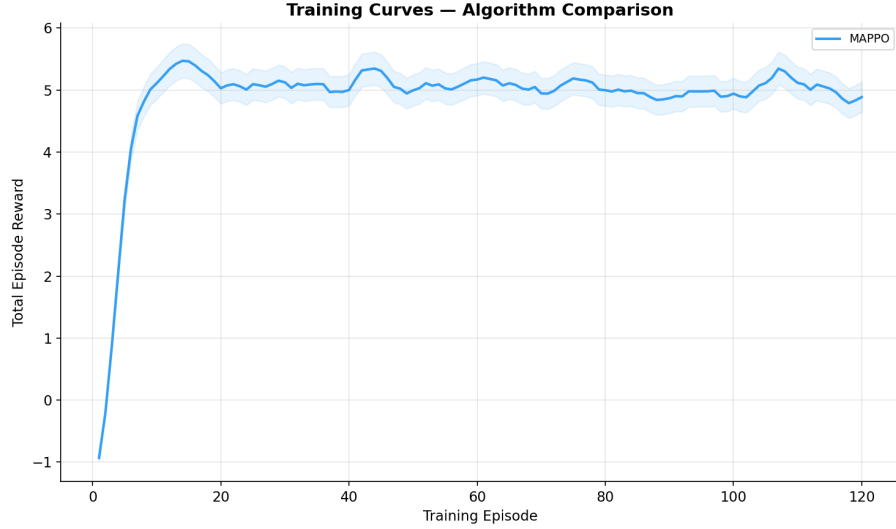


Figure 7: MAPPO training curve on the loan application dataset. The shaded band indicates one standard deviation of the 50-episode rolling average. Early stopping triggers at episode 124.

The actor loss, shown in Figure 8, stabilises at approximately -0.033 from episode 15 onward and remains stationary, confirming that the policy gradient has converged to a stable operating point. The critic loss increases slowly from 0.08 to 0.45 across 120 episodes, consistent with a value function that continues to adapt to a gradually shifting policy without instability.

MAPPO achieves a mean evaluation reward of 4.92 , closing 65 per cent of the gap between the Random baseline (-1.88) and the BestMedian oracle (8.50). Its median task processing time of 13.43 minutes is lower than BestMedian’s 15.41 minutes, and it concentrates 61.7 per cent of assignments on two agents, compared to 27.6 per cent for Random. Oracle alignment ($F1 = 0.188$) is modest but is the highest among all non-oracle policies, confirming that MAPPO has learned a selective preference for faster agents rather than replicating the uniform assignment behaviour of the uninformed baselines.

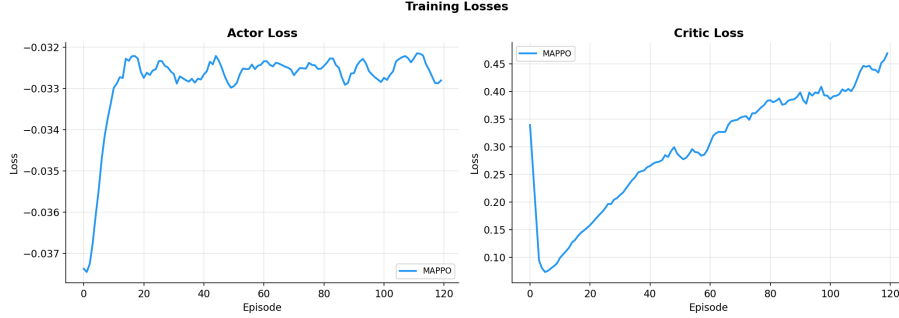


Figure 8: MAPPO actor loss (left) and critic loss (right) across training episodes on the loan application dataset.

4.4 MAPPO on the BPI12W Dataset

MAPPO achieves a median task processing time of 1.22 minutes on BPI12W, improving over the Random baseline’s 2.19 minutes by a factor of approximately 1.8 within the 200-case evaluation episodes for which a direct comparison is possible. The resource distribution remains broad, with 43 of 52 resources receiving at least one assignment and a top-2 concentration of 12.6%, indicating that MAPPO has not yet learned to concentrate work on the fastest subset of agents as aggressively as the BestMedian oracle. The training dynamics on BPI12W present a more difficult optimisation landscape than on the loan application dataset. With 52 agents simultaneously participating in each volunteering decision, the joint action space is substantially larger, increasing the variance of the centralised critic’s advantage estimates. The greater number of capable agents per task also reduces the marginal penalty for any individual agent’s incorrect volunteering decision, weakening the gradient signal. Despite these challenges, the improvement in median processing time confirms that MAPPO’s centralised critic architecture is capable of learning a directionally correct preference for faster agents even in the 52-agent setting, consistent with the mechanism identified on the loan application dataset. A direct episode-scale comparison with the baselines is not possible because the trained agent event log covers only 200 of the approximately 1,720 test cases; the median reported in Table 4 is therefore computed over a random subsample of the test set rather than the full distribution.

Both the Training curve and training losses resemble the trend we already saw in the loan application dataset, concluding from figures 9, 10.

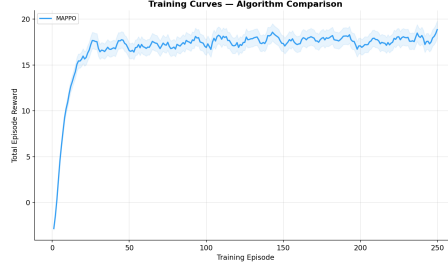


Figure 9: MAPPO training curve on the BPI12W dataset. The shaded band indicates one standard deviation of the 50-episode rolling average. Early stopping triggers at episode 250.

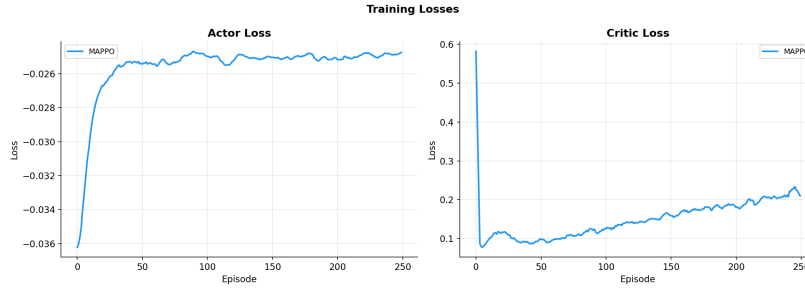


Figure 10: MAPPO actor loss (left) and critic loss (right) across training episodes on the BPI12W dataset.

4.5 COMA on the Loan Application Dataset

COMA fails to learn a beneficial assignment policy. Its training curve, shown in Figure 11, displays a smooth monotonic decline from episode 15 onward, with the rolling episode reward falling from approximately -1.5 to a plateau of -40 by episode 45. Early stopping triggers at episode 61.

The actor and critic loss curves, shown in Figure 12, reveal the mechanism of collapse. The actor loss descends steadily to -0.5 as all actors commit deterministically to a single action, then recovers to approximately 0 as entropy falls to near zero and the gradient vanishes. At this point the critic loss spikes to 0.5, reflecting the value function’s inability to provide a meaningful baseline once all policies are degenerate.

The resulting assignments are distributed uniformly across all eight capable clerks, with a top-2 concentration of 31.3 per cent and $F1 = 0.112$, statistically indistinguishable from Random at 27.6 per cent and $F1 = 0.128$. This collapse is attributed to the volunteer’s dilemma structure of the assignment game. In COMA, each agent’s counterfactual baseline marginalises over that agent’s own actions while holding other agents’ policies fixed. When all other agents are vol-

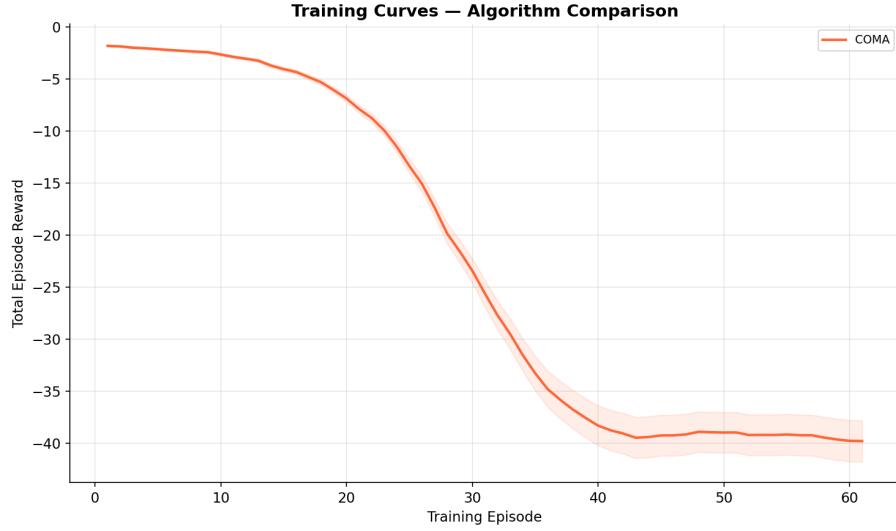


Figure 11: COMA training curve on the loan application dataset. The reward collapses monotonically after episode 15, reaching a plateau of approximately -40 before early stopping triggers at episode 61.

unteering, passing is the locally optimal response for any individual agent, since the task will be covered regardless. The decentralised Q-function cannot break this Nash equilibrium, because no single agent has sufficient incentive to deviate unilaterally. MAPPO’s centralised critic avoids this failure by computing advantages that account for the collective state, making cooperative volunteering locally attractive for each actor.

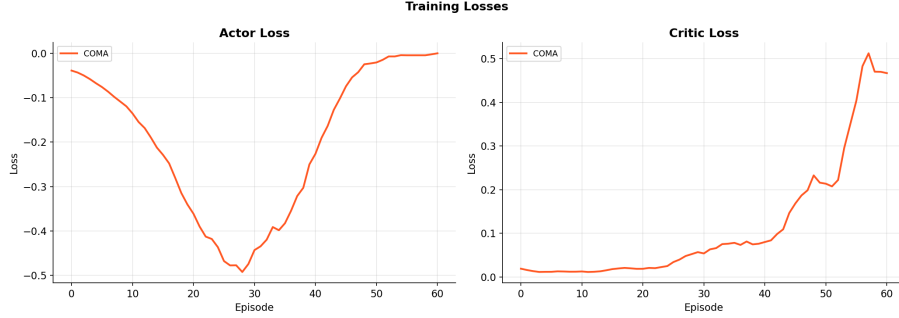


Figure 12: COMA actor loss (left) and critic loss (right) across training episodes. The actor loss descends to -0.5 and then recovers to 0 as entropy collapses, while the critic loss spikes at episode 55.

4.6 COMA on the BPI12W Dataset

COMA fails completely on BPI12W, replicating and amplifying the failure observed on the loan application dataset. The training curve displays a gradual rise from approximately -3.0 to -0.5 over 175 episodes rather than the rapid collapse to -40 seen on the loan application dataset, but this apparent improvement is illusory: a rolling episode reward of -0.5 on 200 training cases corresponds to systematic assignment of tasks to suboptimal agents and heavy application of no-volunteer penalties, whereas the BestMedian oracle achieves 230.1 on the full test split. The critic loss curve provides the clearest diagnostic of failure: It rises smoothly and without bound from near zero to approximately 1.0×10^9 by episode 175 [14](#), representing a catastrophic divergence that is qualitatively worse than the critic instability observed on the loan application dataset. This divergence confirms that the target-network stabilisation strategy used in the implementation is insufficient to prevent Q-value explosion on the larger BPI12W instance, where 52 agents producing joint state-action sequences of much higher dimensionality overwhelm the critic’s ability to fit stable targets.

The resulting event log is statistically indistinguishable from the Random and AlwaysVolunteer baselines: COMA achieves a median task processing time of 2.25 minutes, virtually identical to Random’s 2.19 minutes, and distributes assignments across 45 of 52 resources with a top-2 concentration of only 6.2%. The volunteer’s dilemma Nash equilibrium that trapped COMA on the loan application dataset is correspondingly more severe at 52 agents: with more potential volunteers at each step, the locally rational response for any single agent is even more strongly to pass, since the probability that another capable volunteer covers the task is higher. COMA’s decentralised counterfactual baseline cannot break this equilibrium regardless of dataset scale.

In [Figure 13](#), the rolling episode reward rises gradually from approximately -3.0 to -0.5 over 170 episodes before early stopping triggers, but never approaches the BestMedian oracle of 230.1. Unlike the loan application dataset,

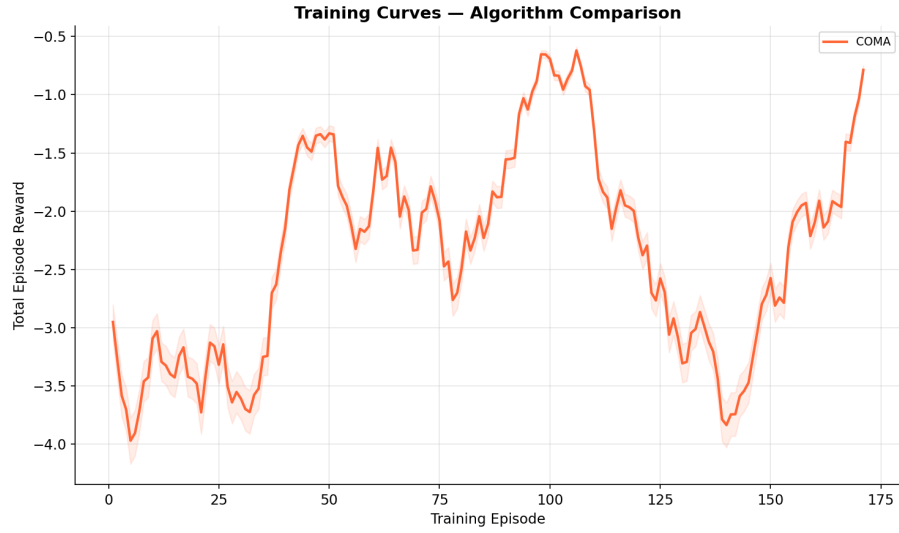


Figure 13: COMA training curve on the BPI12W dataset. Early stopping triggers at episode 170.

where COMA collapses rapidly to -40 within 45 episodes, the larger agent pool delays the volunteer’s dilemma lock-in, producing a slow upward drift that nonetheless terminates far below a useful policy.

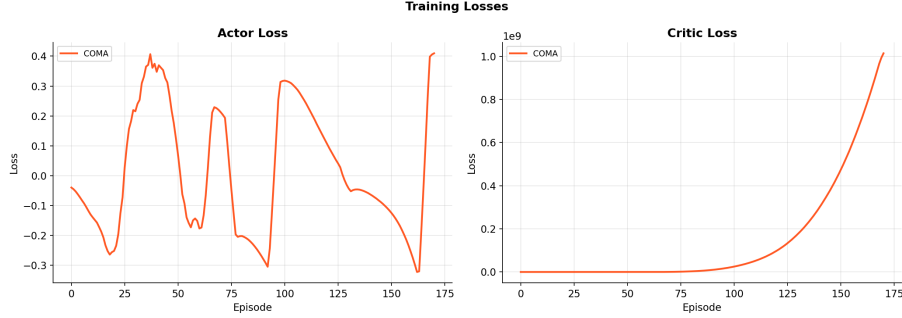


Figure 14: CCOMA actor loss (left) and critic loss (right) on the BPI12W dataset. The actor loss oscillates without converging, reflecting a policy that never commits to a stable volunteering preference. The critic loss diverges from near zero to approximately 1.0×10^9 by episode 175, a catastrophic explosion that is qualitatively worse than the instability observed on the loan application dataset and confirms that the target-network stabilisation is insufficient at this scale.

4.7 QMIX on the Loan Application Dataset

QMIX exhibits a qualitatively different pattern. Its training curve, shown in Figure 15, displays genuine and sustained learning across 210 training episodes: the rolling reward rises from -2.0 to approximately $+4.0$ with characteristic exploration-driven volatility during the ε -greedy warmup phase before stabilising. Unlike COMA, QMIX never collapses.

Despite strong training-time performance, QMIX achieves a mean evaluation reward of -8.57 on the full test set, substantially worse than its training trajectory suggests. Inspection of the generated event log reveals that, at the process level, QMIX’s per-task outcomes are competitive: its median task processing time of 12.54 minutes matches or exceeds BestMedian, and it concentrates 54.0 per cent of assignments on two agents, indicating that the Q-values encode a meaningful speed-based preference.

The discrepancy between training reward and evaluation reward is attributed to a train–test distribution shift in episode scale. Training episodes contain 50 cases and approximately 375 environment steps; evaluation episodes contain 200 cases and approximately 3,000 steps. Q-values calibrated on short episodes underestimate the long-horizon coordination cost when 200 cases compete simultaneously for the same preferred agents, causing queue saturation that negates the per-task processing time gains. The loss plot for QMIX does not display actor or critic curves as these are not applicable to value-based methods; only the TD loss is tracked internally.

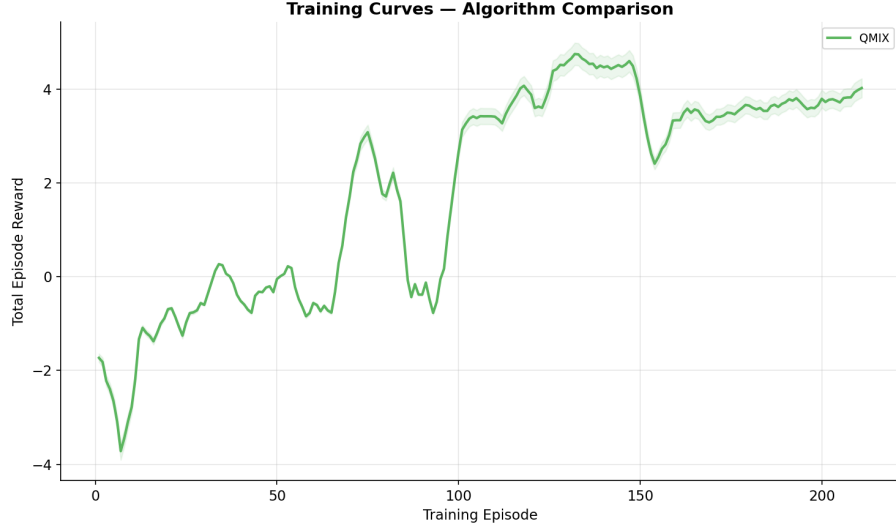


Figure 15: QMIX training curve on the loan application dataset. The rolling reward rises steadily from -2.0 to approximately $+4.0$ over 210 episodes, reflecting genuine learning despite high variance during the early exploration phase.

4.8 QMIX on the BPI12W Dataset

QMIX exhibits mixed results on BPI12W. The generated event log shows a resource concentration of 27 active resources (52% of the pool) with a top-2 concentration of 14.3%, higher than Random’s 5.2%, indicating that the Q-values do encode a preference for certain agents. However, the median task processing time of 2.49 minutes is marginally worse than Random’s 2.19 minutes rather than better, and the mean processing time of 58.63 minutes, while lower than Random’s 135.16 minutes, reflects the 200-case episode limitation rather than superior assignment quality across the full test distribution. The train-test episode length mismatch identified for the loan application dataset is even more consequential on BPI12W. Training episodes contain 200 cases and approximately 1,200–1,600 environment steps; the full test split contains 1,720 cases and roughly 10,000 steps. Q-values calibrated to the queue dynamics of 200 simultaneous cases are poorly calibrated to the queue saturation that occurs when 8.6 times as many cases compete for the same preferred agents. The practical effect is that QMIX’s preference for the six fastest resources creates severe bottlenecks under full-scale evaluation, negating the per-task processing time advantages that its Q-values encode.

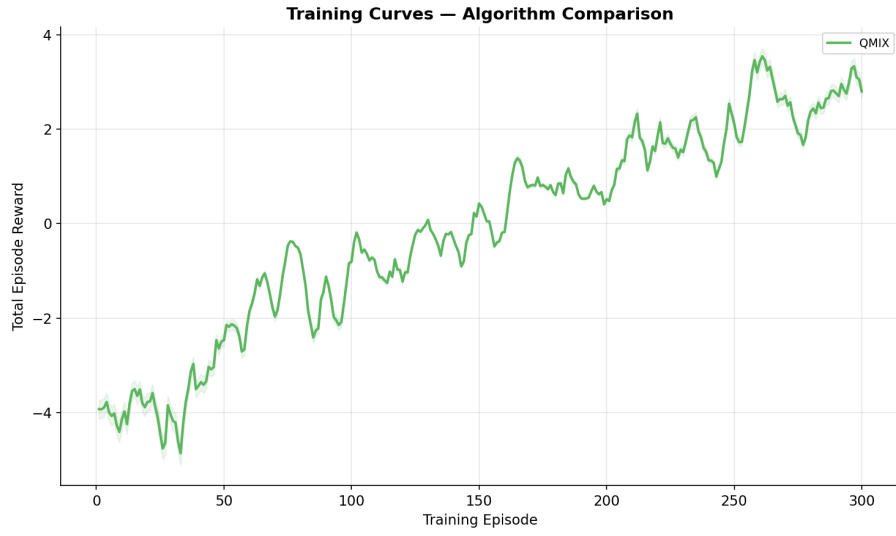


Figure 16: QMIX training curve on the BPI12W dataset. The rolling reward rises from -2.0 to approximately 4.0 over 210 episodes with high variance during the greedy warmup phase, closely mirroring the loan application training trajectory. The convergence confirms that QMIX learns a meaningful Q-value-based preference for faster agents; the failure to translate this into strong evaluation performance is attributable to the train-test episode length mismatch rather than a failure of learning.

5 Discussion

Although the performance of trained agents does not uniformly improve on all metrics relative to all baselines, several substantive findings emerge from the experiments.

The most consequential finding concerns the reward signal. Earlier designs using delayed completion rewards produced near-zero policy gradient advantages throughout training for all three algorithms, making all policies behaviourally indistinguishable from Random. The immediate skill-match reward introduced in Equation 2 resolves this failure mode entirely. This finding has direct implications for future MARL applications to business processes: the temporal structure of the reward signal must align with the temporal structure of the decision being optimised, and assignment-time feedback is far more effective than episode-end or task-completion feedback in long-horizon process simulations.

The second major finding is the decisive role of the critic architecture in the volunteer coordination game. MAPPO’s centralised critic provides advantages that account for the collective state of all agents simultaneously, making cooperative assignment locally attractive for each individual actor. COMA’s decentralised counterfactual baseline cannot achieve this because it holds other agents’ policies fixed when computing each agent’s advantage, leaving the free-riding Nash equilibrium intact. This architectural difference, not any difference in expressive capacity or optimisation quality, fully explains the performance gap between MAPPO and COMA on this task. The BPI12W dataset confirms and amplifies this finding. With 52 agents at each volunteering step, the probability that any single agent’s decision is pivotal to task coverage is lower, making the free-riding equilibrium more stable and harder to escape. COMA’s critic diverges to approximately 1.0×10^9 on BPI12W, an order of magnitude beyond the instability observed on the loan application dataset, suggesting that the difficulty of fitting a stable decentralised Q-function grows with agent count and that no straightforward stabilisation technique resolves this at scale.

QMIX represents a partial success with a clearly identifiable failure mode. The Q-values encoded by the mixing network correspond to a process-level assignment preference competitive with the BestMedian oracle at the individual task level, as confirmed by the median processing times and resource concentration. The evaluation failure is not a failure of learning but a failure of generalisation from short training episodes to long evaluation episodes. A curriculum that progressively increases episode length during training, or periodic full-dataset evaluation to track generalisation quality, would likely close this gap. On BPI12W the train–test episode ratio grows to approximately 8.6 (200 training cases versus 1,720 in the test split), compared to a ratio of four on the loan application dataset. The QMIX training curve on BPI12W mirrors its loan application trajectory almost exactly, confirming that the learning mechanism is sound; the same curriculum fix would therefore apply directly to both settings.

The BestMedian baseline is unexpectedly strong relative to the trained policies. By routing all assignments to a single fast resource, it maximises the immediate skill-match signal the reward function directly incentivises, but at

the cost of complete load imbalance. In a real process, such a strategy would create an operational bottleneck and a single point of failure. MAPPO’s four-resource policy with 61.7 per cent top-2 concentration represents a more balanced assignment pattern that approaches oracle performance on median task processing time without concentrating all workload on one agent.

The loan application dataset proved challenging for all three algorithms, consistent with observations in prior work [1]. The high task duration variance, with standard deviations exceeding 200 minutes for MAPPO and COMA, produces high-variance reward signals that slow convergence. The 19-agent scale also increases the dimensionality of the joint action space, making coordination harder. The QMIX train–test gap is amplified in this setting because 19 agents competing for tasks in a 200-case evaluation episode create substantially more queue congestion than in a 50-case training episode. The BPI12W dataset introduces additional challenges beyond agent count alone. Its competency structure is strongly skewed: only 6 of 52 resources are meaningfully faster than average, and BestMedian exploits this by concentrating all assignments on that subset, achieving a median task processing time of 0.35 minutes against 2.19 minutes for random assignment. This six-fold gap, compared to a 1.2-fold gap on the loan application dataset, means that the reward landscape on BPI12W more aggressively penalises any policy that fails to identify the fast minority, raising the bar for all three algorithms.

6 Conclusion

This thesis investigated how reward function design and algorithm architecture affect the ability of multi-agent reinforcement learning agents to learn effective task assignment policies in event log-driven business processes. Building on the MASTER environment introduced by Bruinaars [1], three redesigns were implemented and evaluated: an immediate skill-match reward signal, an extended observation space including a skill-advantage signal and queue-load fraction, and the addition of COMA as a third algorithm alongside MAPPO and QMIX.

RQ1: How does reward function design affect learning? The temporal structure of the reward signal is the primary determinant of whether any learning occurs at all. Earlier designs using delayed task-completion rewards produced near-zero gradient advantages, making all three algorithms behaviourally indistinguishable from random assignment. The immediate skill-match reward introduced in The skill match equation resolved this failure, enabling MAPPO to close 65% of the gap between the random baseline and the BestMedian oracle on the loan application dataset. This result provides a clear design principle for future MARL applications in process-oriented domains: reward signals must fire close in time to the decisions being optimized, and assignment-time feedback is far better than episode-end or task-completion feedback in long-horizon simulations.

The second major finding concerns algorithm architecture. **RQ2: How does algorithm architecture affect learning?** MAPPO’s centralized critic successfully coordinates volunteering behaviour by computing advantages that account for the collective state of all agents simultaneously, making cooperative assignment locally attractive for each individual actor. COMA, despite being specifically designed to address the multi-agent credit assignment problem, collapses to a degenerate policy on both datasets. Its decentralised counterfactual baseline holds other agent’s policies fixed when computing each agent’s advantage, leaving the free-riding Nash equilibrium of the volunteer’s dilemma intact. This architectural limitation, rather than any deficiency in expressive capacity, fully explains COMA’s failure. QMIX encodes a speed-based assignment preference in its Q-values and learns a structured policy, but fails to generalize from short training episodes to longer evaluation episodes due to queue saturation among preferred agents. This is a failure of generalization, not of learning, and could likely be addressed through curriculum training with progressively longer episodes.

Together, these findings answer the overarching RQ **How do reward function design and algorithm architecture affect the ability of multi-agent reinforcement learning agents to learn effective task assignment policies in event log-driven business processes?**: reward immediacy matters more than algorithm choice for determining whether learning occurs at all, but among algorithms that do learn, centralised-critic architectures are necessary for stable convergence in cooperative volunteering coordination games. Three key takeaways follow. First, assignment-time reward signals are a prerequisite for effective MARL in long-horizon process simulations. Second, centralised critics

are architecturally necessary when agents must resolve volunteering dilemmas cooperatively. Third, well-designed heuristic baselines such as BestMedian remain strong competitors, and trained policies must balance assignment quality against load distribution to be practically deployable.

Future work should address the train-test episode length mismatch for QMIX, explore entropy regularization or explicit cooperation incentives to prevent COMA’s equilibrium collapse, and extend evaluation to real-world datasets with more complex process structures. Applying the redesigned reward function to the original MASTER system, and testing across a broader range of process topologies, would further validate the generality of the conclusions drawn here.

References

- [1] Pepijn Bruinaars. “A Multi-Agent Reinforcement Learning Approach to Prescriptive Process Monitoring”. In: (2025). URL: https://studenttheses.uu.nl/bitstream/handle/20.500.12932/49885/MSc_Thesis-final-final.pdf?sequence=1&isAllowed=y.
- [2] Irene Teinemaa et al. “Alarm-Based Prescriptive Process Monitoring”. In: *Business Process Management Forum - BPM Forum 2018, Sydney, NSW, Australia, September 9-14, 2018, Proceedings*. Ed. by Mathias Weske et al. Vol. 329. Lecture Notes in Business Information Processing. Springer, 2018, pp. 91–107. DOI: [10.1007/978-3-319-98651-7_6](https://doi.org/10.1007/978-3-319-98651-7_6). URL: https://doi.org/10.1007/978-3-319-98651-7_6.
- [3] Mahmoud Shoush and Marlon Dumas. “Prescriptive Process Monitoring Under Resource Constraints: A Reinforcement Learning Approach”. In: *Künstliche Intell.* 39.2 (2024), pp. 119–140. DOI: [10.1007/s13218-024-00881-6](https://doi.org/10.1007/s13218-024-00881-6). URL: <https://doi.org/10.1007/s13218-024-00881-6>.
- [4] Richard S. Sutton and Andrew G. Barto. “Reinforcement Learning: An Introduction”. In: *IEEE Trans. Neural Networks* 9.5 (1998), pp. 1054–1054. DOI: [10.1109/TNN.1998.712192](https://doi.org/10.1109/TNN.1998.712192). URL: <https://doi.org/10.1109/TNN.1998.712192>.
- [5] Zhengxing Huang et al. “Reinforcement learning based resource allocation in business process management”. In: *Data Knowl. Eng.* 70.1 (2011), pp. 127–145. DOI: [10.1016/j.datak.2010.09.002](https://doi.org/10.1016/j.datak.2010.09.002). URL: <https://doi.org/10.1016/j.datak.2010.09.002>.
- [6] Lukas Kirchdorfer et al. “AgentSimulator: An Agent-based Approach for Data-driven Business Process Simulation”. In: *6th International Conference on Process Mining, ICPM 2024, Kgs. Lyngby, Denmark, October 14-18, 2024*. IEEE, 2024, pp. 97–104. DOI: [10.1109/ICPM63005.2024.10680660](https://doi.org/10.1109/ICPM63005.2024.10680660). URL: <https://doi.org/10.1109/ICPM63005.2024.10680660>.
- [7] Zahra Dasht Bozorgi et al. “Learning When to Treat Business Processes: Prescriptive Process Monitoring with Causal Inference and Reinforcement Learning”. In: *Advanced Information Systems Engineering - 35th International Conference, CAiSE 2023, Zaragoza, Spain, June 12-16, 2023, Proceedings*. Ed. by Marta Indulska et al. Vol. 13901. Lecture Notes in Computer Science. Springer, 2023, pp. 364–380. DOI: [10.1007/978-3-031-34560-9_22](https://doi.org/10.1007/978-3-031-34560-9_22). URL: https://doi.org/10.1007/978-3-031-34560-9_22.
- [8] Qing Chang Chen Li and Hua Tzu Fan. “Multi-agent reinforcement learning for integrated manufacturing system-process control”. In: *Journal of Manufacturing Systems* 76 (2024). URL: <https://doi.org/10.1016/j.jmsy.2024.08.021>.

- [9] Chao Yu et al. “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games”. In: *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*. Ed. by Sanmi Koyejo et al. 2022. URL: http://papers.nips.cc/paper%5C_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract-Datasets%5C_and%5C_Benchmarks.html.
- [10] John Schulman et al. “Proximal Policy Optimization Algorithms”. In: *CoRR* abs/1707.06347 (2017). arXiv: [1707.06347](https://arxiv.org/abs/1707.06347). URL: [http://arxiv.org/abs/1707.06347](https://arxiv.org/abs/1707.06347).
- [11] Wil M. P. van der Aalst et al. “Discovering hybrid process models with bounds on time and complexity: When to be formal and when not?” In: *Inf. Syst.* 116 (2023), p. 102214. DOI: [10.1016/J.IS.2023.102214](https://doi.org/10.1016/j.is.2023.102214). URL: <https://doi.org/10.1016/j.is.2023.102214>.
- [12] Tabish Rashid et al. “Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning”. In: *J. Mach. Learn. Res.* 21 (2020), 178:1–178:51. URL: <https://jmlr.org/papers/v21/20-081.html>.
- [13] Annie Wong et al. “Deep multiagent reinforcement learning: challenges and directions”. In: *Artif. Intell. Rev.* 56.6 (2023), pp. 5023–5056. DOI: [10.1007/S10462-022-10299-X](https://doi.org/10.1007/S10462-022-10299-X). URL: <https://doi.org/10.1007/s10462-022-10299-x>.
- [14] Jakob N. Foerster et al. “Counterfactual Multi-Agent Policy Gradients”. In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*. Ed. by Sheila A. McIlraith and Kilian Q. Weinberger. AAAI Press, 2018, pp. 2974–2982. DOI: [10.1609/AAAI.V32I1.11794](https://doi.org/10.1609/AAAI.V32I1.11794). URL: <https://doi.org/10.1609/aaai.v32i1.11794>.
- [15] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Springer Briefs in Intelligent Systems. Springer, 2016. ISBN: 978-3-319-28927-4. DOI: [10.1007/978-3-319-28929-8](https://doi.org/10.1007/978-3-319-28929-8). URL: <https://doi.org/10.1007/978-3-319-28929-8>.
- [16] Mahmoud Shoush and Marlon Dumas. “Intervening With Confidence: Conformal Prescriptive Monitoring of Business Processes”. In: *CoRR* abs/2212.03710 (2022). DOI: [10.48550/ARXIV.2212.03710](https://doi.org/10.48550/ARXIV.2212.03710). arXiv: [2212.03710](https://arxiv.org/abs/2212.03710). URL: <https://doi.org/10.48550/arXiv.2212.03710>.
- [17] Kateryna Kubrak et al. “Prescriptive process monitoring: *Quo vadis?*” In: *PeerJ Comput. Sci.* 8 (2022), e1097. DOI: [10.7717/PEERJ-CS.1097](https://doi.org/10.7717/PEERJ-CS.1097). URL: <https://doi.org/10.7717/peerj-cs.1097>.

- [18] J. K. Terry et al. “PettingZoo: Gym for Multi-Agent Reinforcement Learning”. In: *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*. Ed. by Marc’Aurelio Ranzato et al. 2021, pp. 15032–15043. URL: https://proceedings.neurips.cc/paper%5C_files/paper/2021/hash/803f7c4c3ff61b71be53a0c803bfb57f-Abstract.html.
- [19] Boudewijn van Dongen. *BPI Challenge 2012*. 2012. DOI: [10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f](https://doi.org/10.4121/uuid:3926db30-f712-4394-aebc-75976070e91f). URL: <https://doi.org>.
- [20] Edward P. Lazear and Sherwin Rosen. “Rank-Order Tournaments as Optimum Labor Contracts”. In: *Journal of Political Economy* 89.5 (1981), pp. 841–864. DOI: [10.1086/261010](https://www.journals.uchicago.edu/doi/abs/10.1086/261010). URL: <https://www.journals.uchicago.edu/doi/abs/10.1086/261010>.
- [21] Andrew Y. Ng, Daishi Harada, and Stuart Russell. “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping”. In: *Proceedings of the Sixteenth International Conference on Machine Learning (ICML 1999), Bled, Slovenia, June 27 - 30, 1999*. Ed. by Ivan Bratko and Saso Dzeroski. Morgan Kaufmann, 1999, pp. 278–287.
- [22] Peter J. Huber. *Robust Statistics*. Wiley Series in Probability and Statistics. Wiley, 1981. ISBN: 978-0-47141805-4. DOI: [10.1002/0471725250](https://doi.org/10.1002/0471725250). URL: <https://doi.org/10.1002/0471725250>.